REVISIÓN TÉCNICA / 01

Auditoría del proyecto frente a la rúbrica

Proyecto EDA 2026-2 · Comercio electrónico
Revisión del 29 de octubre de 2026. Base Git: 9174d76 más los cambios locales presentes durante la revisión.

Conclusión: existe una demo ejecutable de diez etapas y un visor local útil para explicarlas. Sin embargo, aún no se acredita el sistema integrado ni la experimentación que exige el curso. Las brechas principales están en la integración observable, las implementaciones avanzadas, el índice B+ y la trazabilidad de los resultados del informe.

Componente de la rúbrica	Peso	Situación observada
Investigación y documentación	6 puntos Grupal	Hay un informe y referencias. Varias afirmaciones y cifras no corresponden al código o carecen de evidencia reproducible.
Implementación del sistema	4 puntos Grupal	Parcial: el lote recorre las llamadas, pero varias estructuras solo demuestran operaciones aisladas. No hay HNSW real ni LSH.
Experimentación y análisis	4 puntos Grupal	El notebook mide convertir IDs a lista y set. Faltan las comparaciones por familia y las métricas de calidad.
Presentación y sustentación	6 puntos Individual	El visor es una fortaleza. No se encontraron diapositivas; la exposición y la defensa oral no pueden evaluarse leyendo el código.

Prioridades antes de entregar

• **P0:** completar similitud aproximada real, corregir el B+, conectar resultados entre etapas y reconstruir el grafo según co-compra.

• **P0:** sustituir las cifras no trazables del informe por mediciones reproducibles y explicar correctamente qué mide cada benchmark.

• **P1:** corregir IDs, validación, prioridades y reproducibilidad; procesar todos los lotes del conjunto seleccionado.

Esta auditoría no asigna una nota. Separa requisitos expresos, hallazgos reproducidos y recomendaciones. No ejecutar un sistema empresarial distribuido no es, por sí mismo, un incumplimiento: el enunciado permite una simulación.

Evidencia: R1: documento del profesor, componentes obligatorios. C1: main.py y src/. D1: informe existente. E1: comprobaciones locales acotadas.

Qué exige la tarea y cómo se interpreta

Requisitos expresos que gobiernan la evaluación

El objetivo general exige que la salida de cada etapa sea una entrada real para etapas posteriores y advierte: *"No se evaluarán implementaciones aisladas sin integración verificable."* La sección de implementación pide una salida observable: recomendación, alerta, ranking o Top-K. La demo debe mostrar prioridades, filtros, similitud frente a un método exacto, grafo y servicio.

Área	Exigencia verificable
Arquitectura	Diez etapas. Etapa 4: B+Tree como índice de disco. Etapa 5: KD-Tree o R-Tree según escenario, con KNN exacto como línea base. Etapas 6, 7 y 8 marcadas [FIJA].
Familias avanzadas	Bloom y Count-Min; una técnica de conjuntos (MinHash+LSH o SimHash); una técnica ANN/métrica permitida; PageRank o Personalized PageRank; algoritmo de comunidades y sus comparaciones.
Escenario B	Prioridades de órdenes y alertas, consultas puntuales y por rango, productos similares y complementarios, sospecha con ventana temporal y verificación, co-compra y servicio.
Grafo del escenario B	Productos como nodos y relación ponderada entre productos comprados en la misma orden.
Escala y mediciones	Datasets de 1K, 10K, 100K, 1M y 10M para etapas [FIJA] y estructuras base reportadas; tiempos, memoria y calidad según familia.
Entrega	Repositorio, README con roles, notebooks, diagrama e informe PDF de 8–12 páginas. Presentación de 10–15 diapositivas, 15 minutos y demo de 3–4 minutos, más preguntas.

Lo que no debe confundirse con una obligación

• No exige Kafka, nube, contenedores, procesamiento paralelo ni cargar 10M registros a la vez en RAM. Parquet y un lector por lotes son decisiones técnicas razonables.

• Sí existe una etapa de preprocesamiento por lotes. No se impone chunksize ni una cantidad concreta por lote. Un experimento de 1K debe evaluar esa población, no solo los primeros 100 sin advertirlo.

• El texto pide los cinco tamaños y las comparaciones. No detalla un diseño factorial para cada caso inviable: documentar las restricciones, el N realmente evaluado y la justificación de recortes.

• Las alternativas exactas sirven como control experimental y para la demo comparativa; no es obligatorio repetir toda consulta productiva con fuerza bruta.

Evidencia: R1: Objetivo general, Arquitectura obligatoria, Bloque de investigación, Opción B y componentes 1–4. Estas interpretaciones no añaden requisitos de infraestructura.

Arquitectura actual e integración pendiente

Recorrido comprobado en la ejecución local

```
Parquet 1K -> FIFO 1000 -> extraer 100 -> limpiar -> heap
                                                      |
                                              lote_priorizado
                                              / / / | \ \ \
                        almacenamiento, B+, KD, filtros, similitud, grafo, Top-K
```

La ejecución de auditoría terminó y dejó **900 eventos pendientes**. El heap quedó vacío, el diccionario almacenó 100 eventos, la clase vectorial recibió 15 vectores y el grafo terminó con 176 nodos y 98 aristas. Son resultados funcionales de esta demo, no medidas de capacidad del sistema a 10M.

Compartir una lista entre etapas es una integración de entrada, pero no acredita por sí solo la integración exigida. El B+ no se consulta; el KD-Tree no resuelve la búsqueda; las respuestas de Bloom/CMS, MinHash y PageRank se imprimen y no modifican una recomendación, una alerta ni el Top-K final. src/pipeline.py está vacío.

Diseño propuesto para obtener evidencia observable

Salida producida	Consumidor posterior y prueba de integración
Evento válido y priorizado	El almacenamiento conserva el evento y los índices registran claves consultables. Un ID permite rastrear su tratamiento.
Consulta puntual o de rango	Sus productos se convierten en candidatos para búsqueda multidimensional o recomendación. La respuesta conserva IDs recuperables en almacenamiento.
Bloom + frecuencia temporal	Produce un indicador de revisión. Una consulta exacta confirma alertas críticas antes de excluir candidatos o actualizar el estado de riesgo.
Candidatos similares	La búsqueda de conjuntos y ANN aporta productos/usuarios candidatos; el grafo agrega evidencia de co-compra e influencia.
Ranking y comunidades	El servicio combina candidatos y señales documentadas para mostrar recomendaciones, complementos, alertas y Top-K.

Estado entre lotes y consultas

Crear las estructuras una vez por ejecución y actualizarlas en cada lote. FIFO y heap contienen trabajo pendiente y pueden vaciarse; almacenamiento, índices, contadores y grafo conservan estado. La consulta de un índice no mueve ni borra el evento del almacenamiento. Definir límites o ventanas para evitar crecimiento de RAM sin control.

Criterio de cierre: un evento identificado deja trazas de las diez etapas y una señal suya cambia o explica una salida del servicio. Incluir pruebas donde alterar la salida de un filtro o del ranking cambie una decisión posterior. No basta con animar diez nodos.

Evidencia: C1: main.py:27–45, 69–74, 85–97, 105–109, 122–139, 150–200 y 210–250. E1: demo_log.txt y evidence.json.

Etapas 0 a 3

Etapa y estado	Qué está implementado	Qué falta para cerrar
0 · Parcial	deque con enqueue/dequeue O(1), capacidad y OverflowError.	Consumir lotes sucesivos; definir espera/rechazo cuando se llena. Describir FIFO real: no hay ring buffer ni cola concurrente implementada.
1 · Parcial	Deduplicación por ID, histórico ordenado y sorted por fecha.	Validar esquema, nulos y tipos; demostrar búsqueda binaria entre lotes. Corregir deduplicación cuadrática del primer lote y documentar el costo del histórico.
2 · Parcial	heapq; menor número primero y comparación de timestamps.	Respetar alertas de pago/fraude y definir urgencia de órdenes. No sobrescribir sin justificación las prioridades de la preparación.
3 · Parcial	dict de almacenamiento y OrderedDict para caché LRU.	Corregir invalidación/actualización de caché. Documentar almacenamiento en RAM y justificar matriz dispersa si aplica o su omisión.

Fallas y límites concretos

• **P1 · Prioridad contradictoria.** El conversor asigna payment_alert=1 y purchase=3, pero main.py:69 reemplaza ambas reglas: compra=1 y todo lo demás=3. Una alerta termina detrás de una compra. Mantener una única política y probar el orden con una alerta, una orden urgente y una vista.

• **P1 · Caché desactualizada, reproducido.** Guardar a={price:1}, consultarlo, reemplazar el valor por {price:2} y consultar otra vez devuelve price=1 desde caché. put_storage debe invalidar o actualizar la entrada cacheada.

• **P1 · Limpieza incompleta.** deduplicate_batch accede directamente a event_id; sort_batch presupone event_time comparable. No se normalizan nulos ni se validan campos aunque el notebook y el informe lo afirmen. Manejar dataset vacío o menor de 100: el dequeue fijo puede introducir None.

• **P2 · Complejidad mal descrita.** En el primer lote, “e_id not in new_ids_to_add” busca en una lista: hasta $O(b^2)$. En lotes posteriores hay búsqueda binaria, pero concatenar y aplicar np.unique al histórico añade trabajo que no es O(log H) total. Usar H para histórico y b para lote al explicar costos.

• **P2 · Empates y valores falsos.** La cola no tiene contador estable para igualdad completa de prioridad/fecha. La caché usa “if data”, por lo que un diccionario vacío se considera ausente; capacidad cero necesita validación.

Aceptación: dos lotes con duplicados históricos; fechas/nulos inválidos; buffer lleno; orden de urgencia verificable; caché que devuelve el dato actualizado y expulsa el menos reciente. No se requiere una base de datos externa para demostrar la etapa 3.

Evidencia: C2: etapa_0:15–27; etapa_1:18–55; etapa_2:12–18 y 29–41; etapa_3:15–42. R1: etapas 0–3 y escenario B. E1: cache_update.

Etapas 4 y 5 e índices de consulta

P0 · El B+ no acredita el índice de disco

La clase permite insertar en nodos en memoria, pero no tiene búsqueda puntual, consulta por rango, serialización, páginas ni reapertura del índice. El mensaje “índices en disco actualizados” de main.py no está respaldado por una escritura de índice.

Fallo estructural reproducido: con t=2, al insertar las claves 0 a 8, un nodo interno conserva las claves [1, 3] y solo dos hijos. Debe tener tres hijos para dos separadores. La división de nodos internos conserva una clave que ya promovió al padre. Completar 100 inserciones sin excepción no demuestra corrección: esta prueba de invariantes falla antes.

```
t = IndexacionPrincipalBPlusTree(t=2)
for key in range(9):
    t.insert(key, key)
# root/0: 2 claves, 2 hijos; se esperaban 3 hijos.
```

Corregir por separado divisiones internas y de hojas, política de separadores y claves repetidas. Usar claves estables con significado (precio, fecha o ID completo), asociarlas a registros y probar contra una referencia ordenada. abs(hash(ID)) % 100000 cambia entre procesos y reduce el espacio a 100.000 claves; no preserva un orden útil de precios o fechas.

Aceptación: invariantes en cada inserción, hojas ordenadas y enlazadas, búsquedas exactas y de rango equivalentes a una referencia, además de guardar y reabrir el índice sin pérdida de resultados.

P1 · KD-Tree construido pero no consultado

El árbol se construye ordenando recursivamente por ejes. knn_exact_baseline no lo recorre: calcula todas las distancias y ordena la lista completa. Para dimensión fija, esa implementación de consulta es O(n log n), no O(n) incluyendo el ordenamiento. La construcción con ordenación en cada nivel es aproximadamente O(n log² n), no una simple inserción O(log n).

Problema	Corrección y comprobación
Coordenadas (precio, user_id)	Un ID arbitrario no expresa semejanza y su escala domina el precio. Definir atributos útiles del producto, normalizarlos y justificar la métrica.
Puntos e IDs desalineados	points admite price=0; data_ids lo excluye. Prueba: dos puntos y un ID, con zip truncando o emparejando mal. Filtrar una sola lista de registros y derivar ambas colecciones.
Vecino trivial	Se consulta un punto ya incluido: devuelve el propio evento. Separar queries, definir empates y excluir el mismo objeto cuando corresponda.

Evidencia: C3: etapa_4:35–80 y main.py:105–109. C4: etapa_5:27–49 y main.py:122–134. E1: bplus_invariant_failure y zero_price_alignment.

Etapa 6 y detección de actividad sospechosa

Qué sí existe

Bloom utiliza varios hashes para marcar pertenencia; Count-Min incrementa una celda por fila y estima el mínimo. Las implementaciones permiten demostrar conceptos con inserciones y consultas. En la demo ambas reciben product_id y se consulta un producto que acaba de insertarse.

P0 · Falta convertir el resumen en una decisión

El profesor exige definir **clave, ventana temporal y umbral de alerta**. No están definidos en el flujo: no hay vencimiento ni ventanas, no se cuenta intensidad de intentos por cuenta/IP/dispositivo y ningún resultado se usa para revisar una transacción o alimentar una recomendación. Consultar únicamente un producto insertado no estima falsos positivos.

Pendiente	Implementación o evidencia de cierre
Clave de detección	Elegir cuenta, dispositivo, medio de pago o IP para sospecha; se puede mantener otro contador de productos para popularidad. Documentar cada propósito.
Ventana y umbral	Ejemplo de diseño: conteo por cuenta en cinco minutos y umbral configurable. El valor es una propuesta, no un número impuesto por la rúbrica. Rotar resúmenes por ventanas o documentar otra estrategia.
Verificación exacta	Un positivo Bloom significa “posiblemente observado”. Verificar una decisión crítica con un conjunto/registro exacto antes de bloquear o confirmar una alerta.
Pruebas de calidad	Separar claves insertadas y ausentes. Medir FP/(FP+TN); comprobar ausencia de falsos negativos con configuración válida; comparar CMS contra frecuencias exactas.
Memoria y parámetros	bit_array es una lista Python, no bits empaquetados. Usar representación compacta o medir honestamente su costo. Publicar capacidad, m, k, width, depth y carga real.

Cómo integrarla sin inventar fraude real

Generar un patrón controlado y etiquetado como simulado, por ejemplo múltiples intentos de la misma cuenta dentro de una ventana. Hacer que el filtro produzca una señal, verificarla y mostrarla en el servicio. La etiqueta is_fraud generada al azar no prueba que el sistema haya detectado fraude; solo describe la simulación de entrada.

Aceptación: un caso normal y otro de intensidad anómala producen salidas distintas; una colisión de Bloom no bloquea automáticamente; se reportan FPR y error de frecuencia frente a referencias exactas, con tiempo y memoria.

Evidencia: R1: Bloque de investigación, párrafo Bloom/CMS y Opción B. C5: etapa_6:8–55; main.py:150–168. D1: p. 8. No hay una ventana ni un umbral ocultos en los notebooks.

Etapa 7 y similitud realmente aproximada

P0 · MinHash no incluye un índice LSH

MinHashLSHSimulator calcula firmas con hashes y estima Jaccard comparando posiciones. No implementa bandas, buckets, inserción de firmas ni recuperación de candidatos por LSH. main.py solo compara las canastas de los dos primeros usuarios, formadas con todos los tipos de evento del lote; no son necesariamente canastas de compra.

Corrección: completar MinHash+LSH o elegir SimHash y justificar la elección permitida. Definir qué es una canasta, el periodo o sesión, el umbral y la población candidata. Evaluar consultas contra Jaccard exacto o la referencia exacta apropiada, usando las mismas entidades.

P0 · La clase HNSW hace fuerza bruta

HNSWApproximateSearch guarda vectores en una lista. search_knn calcula la distancia contra todos y ordena los resultados. No hay capas, enlaces ni navegación; m y ef_construction se almacenan pero no influyen. Con dimensión d, su consulta cuesta O(n·d + n log n), no una búsqueda aproximada con un índice HNSW.

Además, solo se insertan 15 eventos con vectores [precio/100, posición/15]. La segunda coordenada es posición en la lista, no categoría como dice el comentario. La consulta coincide con el primer vector y el vecino devuelto suele ser el propio evento.

Para el sistema	Para el experimento
Elegir e implementar HNSW, IVF o una alternativa métrica permitida. Puede usarse una implementación de referencia si el equipo explica sus internals y lo permite la evaluación.	Ejecutar ANN y KNN exacto sobre los mismos vectores, distancia, queries y K. La búsqueda exacta es el control; no tiene que duplicar cada consulta del servicio.
Construir representaciones útiles y constantes por producto, con normalización y política de actualización; recuperar IDs de productos reales.	Separar construcción y consulta. Medir tiempo, memoria, recall@K, dimensiones, parámetros y número de consultas. Controlar auto-coincidencias y empates.

Aceptación: una consulta de producto genera candidatos usados por el servicio y la demo muestra su comparación con el método exacto. No renombrar fuerza bruta como HNSW. El README menciona hnswlib/datasketch, pero requirements.txt y src no los usan; alinear las tres fuentes.

Evidencia: R1: etapa 7, investigación de conjuntos y ANN, implementación. C6: etapa_7:10–52; main.py:178–200. D1: pp. 9–10. Dependencias: requirements.txt.

Etapas 8 y 9 y salida útil al usuario

P0 · El grafo no es el definido para comercio electrónico

El código usa nx.DiGraph y aristas usuario → producto para todas las interacciones. La tarea define productos como nodos y co-compra en la misma orden como relación ponderada. Repetir add_edge(u,p,weight=1) dos veces deja peso 1: no acumula intensidad. La prueba local lo confirma.

Corrección: identificar órdenes o simular order_id manteniendo su procedencia. user_session no equivale automáticamente a una orden; si se usa como aproximación, explicar y justificar esa decisión. Para compras de A y B en la misma orden, actualizar la relación A–B y su peso. Definir direccionalidad, ventana, productos repetidos y evitar relaciones por simples vistas si se llama co-compra.

Comunidades y ranking

compute_pagerank sí usa NetworkX PageRank. detect_communities ejecuta greedy_modularity_communities sobre un grafo no dirigido; no es Louvain aunque la consola, el README y el informe lo declaren. Seleccionar un algoritmo de la familia indicada, justificarlo y compararlo con una alternativa más simple en el mismo grafo. Greedy puede conservarse como baseline si resulta pertinente.

Faltan el ranking simple por grado/frecuencia, modularidad, tamaños, tiempo y estabilidad entre corridas. NDCG requiere una relevancia de referencia definida; no debe reportarse sin explicar sus etiquetas o criterio. No se exige NDCG como única forma de comparación.

P1 · El servicio actual explica la demo, pero no recomienda

Disponible	Pendiente
Trie por prefijo y Top-K de frecuencia. Visor con etapas, consola, datos acotados, pausas, parada y animaciones explicativas.	Conectar similitud, grafo y filtros a recomendaciones/alertas. Permitir una consulta útil y mostrar qué señales produjeron la respuesta.
Top-K calcula las mayores frecuencias con heapq.nlargest.	Distinguir popularidad por interacciones de ventas: hoy cuenta view, cart y purchase por igual. Definir y rotular la métrica.
Las animaciones explicativas están identificadas como esquemas.	No utilizarlas como evidencia de estructuras que aún faltan. Los tiempos del visor incluyen trazado y snapshots: no son benchmarks, aun descontando la pausa.

Aceptación: una orden con dos productos actualiza una arista; una repetición incrementa su peso; el servicio devuelve un complemento o una alerta explicada con IDs y señales reales. El autocompletado debe responder a un prefijo aportado por el usuario en la demo.

Evidencia: R1: Opción B, grafo de dominio y familias de influencia/comunidades. C7: etapa_8:7–24; main.py:210–250. C8: dashboard.py y dashboard-popover.js.

Datos y reproducibilidad de la preparación

Archivo	Filas verificadas	Tamaño aproximado
ecommerce_1K.parquet	1,000	0.06 MiB
ecommerce_10K.parquet	10,000	0.41 MiB
ecommerce_100K.parquet	100,000	3.50 MiB
ecommerce_1M.parquet	1,000,000	44.70 MiB
ecommerce_10M.parquet	10,000,000	444.07 MiB

Los cinco tamaños sí existen y sus metadatos contienen las filas esperadas. Esto acredita la preparación de datasets; no acredita que se hayan evaluado los algoritmos con esas poblaciones. La auditoría no cargó ni procesó los 10M eventos para medir rendimiento.

P1 · Identificadores que pueden perder eventos válidos

El generador usa str(uuid.uuid4())[:8]: solo 32 bits aleatorios. En el Parquet de 1M se contaron **999.887 IDs distintos**, es decir, 113 filas adicionales comparten identificador. No es una medida de duplicados legítimos de negocio; el generador permite colisiones. La deduplicación y el diccionario pueden descartar o sobrescribir eventos diferentes.

Usar UUID completo o un ID estable con unicidad por fuente y posición. Definir por separado cómo identificar reenvíos legítimos; asignar un UUID nuevo a cada fila repetida del CSV no detecta que el contenido se repitió. Validar unicidad antes de crear los cortes experimentales.

Otras correcciones necesarias

• **Preparación pesada:** lee chunks, pero los acumula y concatena antes de escribir. Escribir Parquet incrementalmente, controlar tipos y evitar iterrows donde se pueda vectorizar. Esto es una mejora de eficiencia, no una exigencia de librería del profesor.

• **Semilla:** el conversor no fija random ni NumPy; uuid4 y Timestamp.now no quedan determinados por random.seed(42). La semilla del notebook no garantiza reproducir los archivos preparados. Guardar configuración, versiones y huellas de los archivos.

• **Fraude simulado:** marca ~1,5% al azar y cambia event_type a payment_alert. Conservar acción original y etiqueta sintética por separado; generar patrones de prueba y declarar la procedencia de IP, prioridad y fraude.

• **Fallback roto:** sin CSV, target_10m=1000 deja cero chunks y falla con “No objects to concatenate”. Con tamaños mayores genera user_id como user_1234, incompatible con int(user_id) en la etapa 5. También pierde el resto si el objetivo no es múltiplo de 1M.

La fuente actual es noviembre; octubre no se usa. No es obligatorio usar ambos CSV. Documentar fuente, recorte y columnas ausentes (por ejemplo orden o disponibilidad) y cómo se completan.

Evidencia: C9: convert_to_parquet.py:36–78, 105–140; main.py:19 y 122–127. E1: metadatos Parquet y count_distinct de 1M. Los CSV completos no se releyeron.

Benchmarks que faltan para acreditar la rúbrica

Qué mide hoy: en 02_benchmarks.ipynb, la región cronometrada convierte df[event_id] a lista y crea un set. No llama a las clases del pipeline. La carga del Parquet ocurre antes del cronómetro y de tracemalloc. El gráfico “del Pipeline” debe renombrarse o sustituirse: esos datos no describen las diez etapas.

Familia	Control y métricas exigidas o necesarias
Bloom	Set exacto; inserción y consulta, tiempo y memoria. FPR sobre claves realmente ausentes; verificación de no falsos negativos. Publicar parámetros y cardinalidad distinta.
Count-Min	Diccionario de conteos; tiempo y memoria, error absoluto/relativo y sobreestimación. Publicar clave, ventana, umbral y distribución.
Conjuntos	MinHash+LSH o SimHash frente al cálculo exacto correspondiente; tiempo, memoria y calidad. Definir universo, umbral, consultas y candidatos.
ANN	Método aproximado frente a KNN exacto/fuerza bruta; tiempo, recall@K y memoria. Mismos vectores, distancia, consultas y exclusión de self-match.
PageRank	Ranking por grado/frecuencia como alternativa simple. Comparar rankings y costos con criterio de relevancia documentado si se usa una métrica supervisada.
Comunidades	Mismo grafo para ambos métodos: modularidad, tiempo, número/tamaños de comunidades y estabilidad. Comparar particiones entre corridas con una medida definida.

Protocolo mínimo reproducible

• Definir N: eventos para ingesta; productos/vectores distintos para ANN; canastas para conjuntos; |V| y |E| para grafos. No llamar “grafo de 10M nodos” a un histórico de 10M eventos sin medirlo.

• Evaluar los cinco datasets para etapas [FIJA] y estructuras base reportadas. Registrar límites de escala, muestras y exclusiones; justificar menos repeticiones cuando corresponda. El texto pide 100 cuando sea viable y contempla restricciones para 10M.

• Separar construcción, actualización y consulta; usar consultas comparables y estados reiniciados por repetición. Reportar media, desviación, repeticiones, plataforma, semilla, parámetros y versiones.

• Guardar cada corrida y su resumen en archivos; graficar log-log con unidades y complejidad esperada. Medir memoria actual y pico; si el backend nativo no queda cubierto por tracemalloc, añadir RSS u otra herramienta y explicar el alcance.

El notebook ya tiene plataforma, semilla, repeticiones y un gráfico: conservar esa estructura, pero cambiar las cargas medidas y generar evidencia por algoritmo. Las salidas guardadas son históricas, no nuevas mediciones realizadas en esta auditoría.

Evidencia: R1: Bloque de investigación y componente 3. N2: 02_benchmarks.ipynb, única celda de código, bloque sample_data/set. D1: pp. 11–12.

Correcciones del informe técnico existente

Se leyó el PDF de docs completo y se inspeccionaron visualmente el diagrama y el gráfico. Tiene **13 páginas totales**, frente a las 8–12 pedidas. Si la institución excluye portada e índice, aclararlo; para evitar ambigüedad, ajustar la entrega o confirmar el criterio. Esta auditoría es un documento distinto del informe del curso.

Ubicación en D1	Discrepancia y corrección
pp. 3–7 y 12	Declara flujo estricto, limpieza/normalización, colas concurrentes, vectorización superior a 10x y B+ en disco. El código no demuestra todo ello. Describir el comportamiento final implementado y medir cualquier aceleración.
p. 8	Publica Bloom 0,90 MB vs set 12,50 MB, throughput y FPR 0,82%. No hay experimento Bloom/set en el notebook ni datos crudos para reproducir esas cifras. Añadir scripts, parámetros y consultas ausentes o retirarlas.
p. 9	Declara LSH con 100K canastas y aceleración 139,5x. No existe índice LSH ni experimento de canastas. Completar la técnica y medir. La consulta LSH no es O(1) total garantizado: depende de bandas, buckets y candidatos.
pp. 9–10	Declara HNSW de 128 dimensiones, recall 96,4% y latencias/QPS. La demo usa 15 vectores de 2 dimensiones y fuerza bruta. Falta definir corpus, consultas, K, parámetros y mediciones correspondientes.
pp. 10–11	Declara Louvain, Q=0,78 y PageRank NDCG@20=0,89 en grafos grandes. El código usa greedy y un grafo pequeño; no calcula Q, NDCG ni compara rankings/particiones. Añadir evidencia o retirar resultados.
pp. 11–13	La tabla reproduce los valores del benchmark lista/set. No demuestra rendimiento de las etapas. La conclusión “inferiores a 100 ms” para 10M no se sostiene con esa medición: la tabla reporta 40.065,46 ms para otra operación.

Estado de las cifras: no verificables con los archivos revisados. Esto no determina si provienen de otro experimento externo al repositorio. Si existen, incorporarlos con sus resultados crudos y condiciones; si son literatura o ejemplos, identificarlos explícitamente como tales.

Documentación y referencias

Para etapas 0–5: función, costos reales y alternativas. Para las cinco familias avanzadas: fundamento, funcionamiento interno, complejidad, usos, límites, trade-offs y comparaciones reproducibles. Corregir Broder: el título de la referencia del profesor es “On the resemblance and containment of documents”. Aclarar publicación final 2020 y versión previa 2018 de HNSW; no mezclar versiones. Completar referencias de PageRank y del algoritmo de comunidades realmente elegido.

Evidencia: D1: Proyecto_Final_GrupoEDA.pdf, pp. 3–13. R1: componentes 1 y 3 y lista de referencias. N2: salidas guardadas y código. No se han auditado fuentes externas no incluidas.

Pruebas realizadas y límites de la revisión

Comprobación acotada	Resultado observado	Implicación
Demo real, sin visor	1K cargados, 100 procesados, 900 pendientes; 15 vectores.	La ejecución completa llamadas, no todo el dataset.
B+ con claves 0…8, t=2	Nodo interno con 2 claves y 2 hijos, en lugar de 3 hijos.	Falla una invariante aunque insertar no siempre lance una excepción.
Actualización de caché	storage price=2, caché devuelve price=1.	Dato obsoleto tras reemplazo en almacenamiento.
Arista repetida dos veces	Peso final=1.	No acumula frecuencia de interacción.
IDs de 1M	999.887 distintos de 1.000.000.	113 filas excedentes comparten identificador. Revisar colisiones y no tratarlas como duplicados válidos.
Fallback sintético a 1K	ValueError: No objects to concatenate.	No sirve reducir el objetivo sin corregir la generación sintética.
user_id sintético	int(user_1234) genera ValueError.	Formato incompatible con la etapa 5.
Precio cero	Dos puntos y un ID con los filtros actuales.	Riesgo de truncamiento/desalineación en zip.

Qué cubren las pruebas existentes

tests/test_dashboard.py contiene dos pruebas de observación del visor, cancelación y reinicio; ambas pasaron en esta revisión. No son una validación de B+, ANN, LSH, filtros ni de los requisitos de negocio. Añadir invariantes de árbol, consultas contra referencia exacta, deduplicación entre lotes, alertas verificadas, consistencia de caché y co-compra con pesos.

Cómo reproducir la evidencia de esta auditoría

```
.venv\Scripts\python.exe tmp/pdfs/audit_checks.py
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

audit_checks.py lee el informe, verifica metadatos y ejecuta casos pequeños. El caso de fallback usa mocks y no crea un dataset sintético masivo. La demo usa el Parquet existente. Los resultados están en tmp/pdfs/evidence.json y demo_log.txt; el script no corrige código de producción.

Límites: no se ejecutaron benchmarks de 10M ni todos los algoritmos a todas las escalas; no se inspeccionaron íntegramente los CSV; la unicidad se contó en 1K y 1M. No se verificaron la entrega al campus, las contribuciones fuera del Git local ni la defensa oral. Las afirmaciones no ejecutadas se marcan como revisión estática.

Evidencia: E1: audit_checks.py y evidence.json generados el 29/10/2026. T1: tests/test_dashboard.py. Los tiempos de esta revisión no son resultados de benchmark del curso.

Plan de cierre por prioridad y responsable

Responsables propuestos según la tabla del README; la asignación no acredita autoría ni reemplaza el historial de contribuciones. P0 bloquea demostrar un requisito central; P1 es necesario para una entrega consistente; P2 mejora robustez o claridad.

Orden	Trabajo y criterio de cierre	Referencia de responsable
1 · P0	Definir contratos de salida y una historia de negocio que use las diez etapas. Un evento debe afectar una alerta, consulta o recomendación verificable.	Equipo completo
2 · P0	Corregir B+, agregar búsqueda/rangos y persistencia. Invariantes y reapertura equivalentes a referencia.	Erick Poma Etapas 3–4
3 · P0	Implementar técnica de conjuntos completa y ANN real con queries y atributos útiles; conservar exactos para comparar.	Guillermo Calderón Etapas 7–9
4 · P0	Definir clave, ventana, umbral y verificación. La señal del filtro debe consumirse en una etapa posterior.	Mentor Naquiche Etapas 5–6
5 · P0	Construir co-compra entre productos y pesos; elegir comunidades, comparar y usar ranking/candidatos en servicio.	Guillermo + equipo
6 · P1	Lector por lotes, estado persistente entre lotes, validación, prioridades coherentes e IDs sin colisiones. Datos reproducibles y formatos compatibles.	Jesús Delgado Etapas 0–2; apoyo del equipo
7 · P1	Consultar KD-Tree con atributos válidos; corregir listas desalineadas. Caché consistente tras actualización.	Mentor y Erick
8 · P0	Benchmarks por familia y tamaños, métricas de calidad y memoria actual/pico; guardar corridas, gráficos y configuración.	Cada responsable mide sus etapas
9 · P1	Reescribir resultados/conclusiones y complejidades conforme al código; eliminar o documentar cifras no trazables. Ajustar extensión.	Equipo completo
10 · P1	Versionar archivos locales, fijar dependencias, documentar datos y preparar diapositivas/demo.	Equipo completo

Orden práctico de validación

Primero corregir algoritmos y probar casos pequeños; después integrar una demo verificable de 1K completo. Continuar con 10K y 100K, medir las familias y escalar a 1M/10M con restricciones registradas. Solo entonces consolidar las cifras del informe y ensayar la presentación. Publicar la web no es una exigencia de la tarea ni sustituye estas correcciones.

Evidencia: Roles: README, tablas de arquitectura y contribución. Prioridades y secuencia: recomendaciones de auditoría, no reparto adicional impuesto por el profesor.

Entrega y sustentación verificables

Checklist final de cumplimiento

Evidencia para entregar	Estado al revisar
Código, README, notebooks e informe en repositorio	Existen archivos base. Visor, animaciones y tests nuevos estaban sin seguimiento en git status; deben incorporarse conscientemente antes de compartir.
Instalación y acceso a datos	requirements sin versiones; hnswlib/datasketch declarados pero no usados. data/ está ignorado; documentar cómo obtener una muestra y generar cortes correctos. 2019-Oct.csv está sin seguimiento: no subir los CSV grandes por accidente.
Diagrama fiel a las dependencias	Existe PNG y visor. El diagrama lineal del informe no demuestra el consumo real de salidas; actualizarlo junto con la integración final.
Informe de 8–12 páginas	Existe PDF de 13 páginas totales; ajustar o confirmar criterio de portada/índice. Contenido, tablas y conclusiones requieren las correcciones de esta auditoría.
Presentación y demo	No se encontraron diapositivas en los archivos listados. Preparar 10–15, exposición de 15 min, demo de 3–4 min y preguntas; verificar entrega al campus por separado.
Contribución individual	README sí asigna al menos dos etapas a cada integrante. Git local tiene cinco commits de una identidad de autor; no permite acreditar por sí solo la distribución declarada. No equivale a concluir que otros no contribuyeron.

Preguntas que cada integrante debería poder responder

¿Qué salida tuya consume otra etapa? ¿Qué queda en RAM y qué en disco? ¿Por qué un positivo Bloom necesita verificación? ¿Qué error introduce CMS? ¿Dónde están LSH y HNSW? ¿Qué significa recall@K? ¿Qué es una orden en el dataset? ¿Cómo validas B+ y qué cuenta realmente el benchmark? Las preguntas del profesor pueden cubrir cualquier etapa, no solo la asignada.

Índice de fuentes y alcance

R1. Downloads/Proyecto EDA - 2026-2.docx: objetivo, arquitectura, investigación, Opción B, componentes, entregables y referencias.
D1. docs/Proyecto_Final_GrupoEDA.pdf: 13 páginas leídas.
C1–C9. main.py, los módulos de src, requirements.txt, README y archivos dashboard.*.
N1–N2. notebooks/01_pipeline_demo.ipynb y 02_benchmarks.ipynb: código y salidas guardadas.
T1. tests/test_dashboard.py. **E1.** tmp/pdfs/audit_checks.py, evidence.json y demo_log.txt.
Git. HEAD 9174d76 y cambios locales revisados; no se revisó un remoto ni contribuciones externas.

Resultado esperado del cierre: una demo que explique decisiones reales, algoritmos correctamente identificados, experimentos repetibles y un informe que solo afirme lo que el código y la evidencia sostienen.